

MLOPS LAB GUIDE

Workflow Automation, Pipeline Orchestration & Scheduling / Monitoring

Hands-On Lab Guide

Module: MLOps Foundations — Automation & Ops Layer

Cloud Stack: Amazon Web Services (AWS)

Audience Level: Intermediate (comfortable with Python, CLI, Docker)

How to Use This Guide

This guide continues from the Deployment & Serving lab guide and contains three progressive, hands-on labs covering how ML workflows get automated, orchestrated, and kept healthy in production. Complete the labs in order — Lab 2 reuses concepts from Lab 1, and Lab 3 monitors artifacts produced by Lab 2.

Cost Warning: Step Functions, Lambda, and EventBridge are pay-per-use and inexpensive for lab purposes. SageMaker Pipeline steps that spin up training/processing instances DO incur per-minute charges — always run the Cleanup section at the end of each lab.

Labs in this Guide

1. Lab 1 — Workflow Automation with AWS Step Functions
2. Lab 2 — Pipeline Orchestration with Amazon SageMaker Pipelines
3. Lab 3 — Scheduling & Monitoring (EventBridge, CloudWatch, SageMaker Model Monitor)

Lab 1: Workflow Automation with AWS Step Functions

Duration	90 minutes
Cloud Services	Step Functions, Lambda, S3, IAM, CloudWatch
Difficulty	Intermediate
Deliverable	Working AWS deployment + screenshot/output evidence submitted per instructions below

Learning Objectives

- Break an ML workflow (prep → train → evaluate → conditional deploy) into independent, automatable steps
- Author an AWS Step Functions state machine using Amazon States Language (ASL)
- Implement conditional branching so deployment only happens if model quality passes a threshold
- Add automatic retries and error handling to make the workflow resilient to transient failures

Prerequisites

- AWS CLI configured; IAM permissions for Step Functions, Lambda, and S3
- Basic JSON familiarity
- (Optional) Artifacts from the Deployment & Serving guide, Lab 1

Architecture Overview

A Step Functions state machine calls four Lambda functions in sequence: PrepareData → TrainModel → EvaluateModel → (Choice) → DeployModel or NotifyFailure. Each Lambda is a lightweight stand-in for a real ML step so the lab focuses on orchestration mechanics rather than ML code.

Step-by-Step Instructions

Step 1: Write four Lambda functions

prepare_data.py — simulates data validation and returns a dataset pointer:

```
def handler(event, context):
    return {'dataset_uri': 's3://mlops-lab-bucket/data/clean.csv', 'rows': 10000}
```

train_model.py — simulates training and returns a model artifact pointer:

```
def handler(event, context):
    return {'model_uri': 's3://mlops-lab-bucket/models/candidate.tar.gz',
        'dataset_uri': event['dataset_uri']}
```

evaluate_model.py — simulates evaluation, returning an accuracy score:

```
import random
def handler(event, context):
    accuracy = round(random.uniform(0.80, 0.97), 4)
    return {'model_uri': event['model_uri'], 'accuracy': accuracy}
```

deploy_model.py — simulates deployment:

```
def handler(event, context):
    return {'status': 'DEPLOYED', 'model_uri': event['model_uri'], 'accuracy':
event['accuracy']}
```

Step 2: Deploy all four Lambda functions

```
for fn in prepare_data train_model evaluate_model deploy_model; do
    zip ${fn}.zip ${fn}.py
    aws lambda create-function --function-name $fn \
        --runtime python3.11 --handler ${fn}.handler \
        --zip-file fileb://${fn}.zip \
        --role arn:aws:iam::<ACCOUNT_ID>:role/lambda-basic-role --timeout 30
done
```

Step 3: Author the state machine (Amazon States Language)

Save as workflow.asl.json:

```
{
  "Comment": "ML training & conditional deployment workflow",
  "StartAt": "PrepareData",
  "States": {
    "PrepareData": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:<ACCOUNT_ID>:function:prepare_data",
      "Retry": [{ "ErrorEquals": ["States.TaskFailed"], "IntervalSeconds": 5,
"MaxAttempts": 2, "BackoffRate": 2.0}],
      "Next": "TrainModel"
    },
    "TrainModel": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:<ACCOUNT_ID>:function:train_model",
      "Retry": [{ "ErrorEquals": ["States.TaskFailed"], "IntervalSeconds": 10,
"MaxAttempts": 2, "BackoffRate": 2.0}],
      "Next": "EvaluateModel"
    },
    "EvaluateModel": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:<ACCOUNT_ID>:function:evaluate_model",
      "Next": "QualityGate"
    },
    "QualityGate": {
      "Type": "Choice",
      "Choices": [
        { "Variable": "$.accuracy", "NumericGreaterThanEquals": 0.90, "Next":
"DeployModel" }
      ],
      "Default": "NotifyFailure"
    },
    "DeployModel": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:<ACCOUNT_ID>:function:deploy_model",
      "End": true
    },
    "NotifyFailure": {
      "Type": "Fail",
      "Cause": "Model accuracy below deployment threshold"
    }
  }
}
```

```
}
}
}
```

Step 4: Create the state machine

```
aws stepfunctions create-state-machine \
  --name ml-training-workflow \
  --definition file://workflow.asl.json \
  --role-arn arn:aws:iam::<ACCOUNT_ID>:role/StepFunctionsExecutionRole \
  --type STANDARD
```

Step 5: Execute the workflow and inspect the run

```
aws stepfunctions start-execution \
  --state-machine-arn arn:aws:states:us-east-1:<ACCOUNT_ID>:stateMachine:ml-
training-workflow \
  --input '{}'
```

```
aws stepfunctions describe-execution --execution-arn <execution-arn>
```

In the Step Functions console, open the execution and view the visual graph — each state lights up green (succeeded) or red (failed) as it runs, and you can click any state to see its exact input/output JSON.

Step 6: Prove the quality gate works both ways

Because `evaluate_model.py` returns a random accuracy between 0.80 and 0.97, run the execution 5–10 times. You should see some runs take the `DeployModel` branch and others take the `NotifyFailure` branch — confirming the `Choice` state genuinely branches on data, not just on success/failure of a step.

Validation Checklist

- State machine executes end-to-end without a schema error
- At least one execution reaches `DeployModel` and at least one reaches `NotifyFailure` across repeated runs
- The visual graph in the console correctly highlights the path each execution took
- A deliberately broken Lambda (e.g., temporarily raise an exception in `train_model`) demonstrates the `Retry` policy attempting twice before the execution fails

Troubleshooting

Issue	Likely Cause / Fix
States.Runtime error at start	Validate <code>workflow.asl.json</code> — trailing commas or mismatched braces are the most common cause; use the Step Functions console's built-in linter
Task fails with AccessDenied	<code>StepFunctionsExecutionRole</code> needs <code>lambda:InvokeFunction</code> permission on each function ARN referenced in the definition
Choice always takes Default branch	Check the exact JSON key name — <code>\$.accuracy</code> must match the key returned by <code>evaluate_model.py</code> exactly, including case

Challenge Exercise

Add a `Parallel` state that runs two independent evaluation checks (e.g., accuracy AND fairness/bias score) concurrently before the `QualityGate`, and only proceed to `DeployModel` if both pass.

Cleanup

```
aws stepfunctions delete-state-machine --state-machine-arn arn:aws:states:us-east-1:<ACCOUNT_ID>:stateMachine:ml-training-workflow
for fn in prepare_data train_model evaluate_model deploy_model; do
    aws lambda delete-function --function-name $fn
done
```

Lab 2: Pipeline Orchestration with SageMaker Pipelines

Duration	120 minutes
Cloud Services	SageMaker Pipelines, Processing, Training, Model Registry, S3
Difficulty	Intermediate
Deliverable	Working AWS deployment + screenshot/output evidence submitted per instructions below

Learning Objectives

- Build a native, ML-purpose-built pipeline using the SageMaker Python SDK instead of generic orchestration
- Chain Processing, Training, Evaluation, and Conditional Model Registration steps into one versioned DAG
- Parametrize a pipeline so the same definition can be re-run with different inputs
- Register only models that pass a quality gate into the SageMaker Model Registry

Prerequisites

- Python environment with sagemaker>=2.100, boto3, pandas installed
- SageMaker execution role with S3 and SageMaker permissions
- Familiarity with the churn dataset from the Deployment & Serving guide (or any tabular CSV with a binary label)

Why a Purpose-Built Orchestrator?

Step Functions (Lab 1) is a general-purpose orchestrator — great for gluing together arbitrary AWS services. SageMaker Pipelines is purpose-built for ML: it natively understands training jobs, processing jobs, model lineage, caching of unchanged steps, and integrates directly with the Model Registry and SageMaker Studio's pipeline DAG viewer. Many teams use both — Pipelines for the ML-specific DAG, Step Functions or EventBridge to trigger it and stitch it into wider business workflows.

Architecture Overview

ProcessingStep (clean & split data) → TrainingStep (XGBoost) → ProcessingStep (evaluate on held-out set) → ConditionStep (accuracy ≥ threshold?) → RegisterModel (only on the pass branch).

Step-by-Step Instructions

Step 1: Define pipeline parameters

Save as build_pipeline.py and build it incrementally as you go:

```
import sagemaker
from sagemaker.workflow.parameters import ParameterString, ParameterFloat

sess = sagemaker.Session()
role = 'arn:aws:iam::<ACCOUNT_ID>:role/SageMakerExecutionRole'
bucket = '<BUCKET>'

input_data = ParameterString(name='InputData',
                              default_value=f's3://{bucket}/data/raw/churn.csv')
accuracy_threshold = ParameterFloat(name='AccuracyThreshold', default_value=0.85)
```

Step 2: Add a Processing step to clean and split data

```
from sagemaker.sklearn.processing import SKLearnProcessor
from sagemaker.processing import ProcessingInput, ProcessingOutput
from sagemaker.workflow.steps import ProcessingStep

sklearn_processor = SKLearnProcessor/framework_version='1.2-1', role=role,
instance_type='ml.m5.large', instance_count=1)

step_process = ProcessingStep(
    name='PreprocessChurnData',
    processor=sklearn_processor,
    inputs=[ProcessingInput(source=input_data,
destination='/opt/ml/processing/input')],
    outputs=[
        ProcessingOutput(output_name='train', source='/opt/ml/processing/train'),
        ProcessingOutput(output_name='validation',
source='/opt/ml/processing/validation'),
        ProcessingOutput(output_name='test', source='/opt/ml/processing/test'),
    ],
    code='preprocess.py',
)
```

preprocess.py is a plain pandas/sklearn script that reads /opt/ml/processing/input, does a train/val/test split, and writes CSVs to the three output directories — identical to a script you'd run locally, just executed inside a managed container.

Step 3: Add a Training step

```
from sagemaker.estimator import Estimator
from sagemaker.inputs import TrainingInput
from sagemaker.workflow.steps import TrainingStep
from sagemaker import image_uris

image = image_uris.retrieve('xgboost', sess.boto_region_name, version='1.7-1')
xgb_estimator = Estimator(image, role, instance_count=1,
instance_type='ml.m5.large',
    output_path=f's3://{bucket}/pipeline-models')
xgb_estimator.set_hyperparameters(objective='binary:logistic', num_round=100,
max_depth=5)

step_train = TrainingStep(
    name='TrainChurnModel',
    estimator=xgb_estimator,
    inputs={
        'train':
TrainingInput(s3_data=step_process.properties.ProcessingOutputConfig.Outputs['train
'].S3Output.S3Uri, content_type='text/csv'),
        'validation':
TrainingInput(s3_data=step_process.properties.ProcessingOutputConfig.Outputs['valid
ation'].S3Output.S3Uri, content_type='text/csv'),
    },
)
```

Step 4: Add an Evaluation step

```
from sagemaker.processing import ScriptProcessor
from sagemaker.workflow.properties import PropertyFile

eval_processor = ScriptProcessor(image_uri=image, command=['python3'], role=role,
```

```

instance_type='ml.m5.large', instance_count=1)

evaluation_report = PropertyFile(name='EvaluationReport', output_name='evaluation',
path='evaluation.json')

step_eval = ProcessingStep(
    name='EvaluateChurnModel',
    processor=eval_processor,
    inputs=[

ProcessingInput(source=step_train.properties.ModelArtifacts.S3ModelArtifacts,
destination='/opt/ml/processing/model'),

ProcessingInput(source=step_process.properties.ProcessingOutputConfig.Outputs['test
'].S3Output.S3Uri, destination='/opt/ml/processing/test'),
    ],
    outputs=[ProcessingOutput(output_name='evaluation',
source='/opt/ml/processing/evaluation')],
    code='evaluate.py',
    property_files=[evaluation_report],
)

```

evaluate.py loads the model artifact, scores it against the held-out test set, and writes {"accuracy": 0.93} to /opt/ml/processing/evaluation/evaluation.json.

Step 5: Add the Condition step and conditional registration

```

from sagemaker.workflow.condition_step import ConditionStep
from sagemaker.workflow.conditions import ConditionGreaterThanOrEqualTo
from sagemaker.workflow.functions import JsonGet
from sagemaker.workflow.model_step import ModelStep
from sagemaker.model import Model

model = Model(image_uri=image,
model_data=step_train.properties.ModelArtifacts.S3ModelArtifacts, role=role)

register_args = model.register(
    content_types=['text/csv'], response_types=['text/csv'],
    inference_instances=['ml.m5.large'], transform_instances=['ml.m5.large'],
    model_package_group_name='ChurnModelGroup',
    approval_status='PendingManualApproval',
)
step_register = ModelStep(name='RegisterChurnModel', step_args=register_args)

cond_gte = ConditionGreaterThanOrEqualTo(
    left=JsonGet(step_name=step_eval.name, property_file=evaluation_report,
json_path='accuracy'),
    right=accuracy_threshold,
)

step_cond = ConditionStep(
    name='CheckAccuracyThreshold',
    conditions=[cond_gte],
    if_steps=[step_register],
    else_steps=[],
)

```

Step 6: Assemble and upsert the pipeline

```
from sagemaker.workflow.pipeline import Pipeline

pipeline = Pipeline(
    name='ChurnModelPipeline',
    parameters=[input_data, accuracy_threshold],
    steps=[step_process, step_train, step_eval, step_cond],
    sagemaker_session=sess,
)

pipeline.upsert(role_arn=role)
execution = pipeline.start()
execution.wait()
print(execution.list_steps())
```

Step 7: Inspect the pipeline DAG and lineage

Open SageMaker Studio → Pipelines → ChurnModelPipeline to see the visual DAG update in real time as each step runs. Click any step to see its exact inputs, outputs, logs, and — for the training step — the full lineage graph back to the source data. Check Model Registry → ChurnModelGroup to confirm a new model version appears only when the run's accuracy cleared the threshold.

Step 8: Re-run with different parameters (no code changes)

```
execution2 = pipeline.start(parameters={'AccuracyThreshold': 0.97})
execution2.wait()
# With a stricter threshold, expect CheckAccuracyThreshold to skip
RegisterChurnModel
```

Validation Checklist

- Pipeline executes all four steps to completion in Studio's DAG view
- A new model version appears in ChurnModelGroup after a passing run
- Re-running with a higher AccuracyThreshold correctly skips registration
- Re-running the pipeline unchanged reuses cached results for steps whose inputs didn't change (visible as 'Cached' in the console)

Troubleshooting

Issue	Likely Cause / Fix
ClientError: model_package_group does not exist	Create it once: <code>aws sagemaker create-model-package-group --model-package-group-name ChurnModelGroup</code>
JsonGet returns a ParseError	Confirm <code>evaluate.py</code> writes valid JSON with the exact key referenced in <code>json_path</code> , and that the <code>PropertyFile</code> path matches the <code>ProcessingOutput</code> 's source directory
Pipeline step stuck in Starting	Check the underlying Processing/Training job in the SageMaker console directly — pipeline steps are thin wrappers around ordinary jobs and surface the same CloudWatch Logs

Challenge Exercise

Add a fifth step, using a Lambda step (`sagemaker.workflow.lambda_step.LambdaStep`), that posts a Slack/SNS notification with the run's accuracy and registration decision — bridging SageMaker Pipelines back into the kind of general-purpose automation you built in Lab 1.

Cleanup

```
aws sagemaker delete-pipeline --pipeline-name ChurnModelPipeline
aws sagemaker delete-model-package-group --model-package-group-name ChurnModelGroup
# Note: delete-model-package-group requires deleting individual package versions
first if any exist
```

Lab 3: Scheduling & Monitoring

Duration	90 minutes
Cloud Services	EventBridge, CloudWatch, SNS, SageMaker Model Monitor
Difficulty	Intermediate
Deliverable	Working AWS deployment + screenshot/output evidence submitted per instructions below

Learning Objectives

- Schedule recurring pipeline runs using Amazon EventBridge (cron and rate expressions)
- Build a CloudWatch dashboard and alarms to watch a live model endpoint
- Route alarm breaches to a human via SNS email/SMS notification
- Enable SageMaker Model Monitor to automatically detect data drift against a training baseline

Prerequisites

- Completion of Lab 2 (ChurnModelPipeline) and, ideally, the Deployment & Serving guide (a live endpoint with Data Capture enabled)
- IAM permissions for EventBridge, SNS, and CloudWatch

Architecture Overview

Two independent flows: (1) a time-based EventBridge rule that re-triggers ChurnModelPipeline weekly to retrain on fresh data; (2) a metrics-based flow where CloudWatch watches endpoint health metrics and Model Monitor watches statistical drift, both fanning out alerts through one SNS topic.

Part A — Scheduling Pipeline Runs

Step 1: Create an SNS topic for all alerts

```
aws sns create-topic --name mlops-alerts
aws sns subscribe --topic-arn arn:aws:sns:us-east-1:<ACCOUNT_ID>:mlops-alerts \
  --protocol email --notification-endpoint you@example.com
# Confirm the subscription via the email you receive
```

Step 2: Create a Lambda that starts the pipeline

```
# trigger_pipeline.py
import boto3
sm = boto3.client('sagemaker')

def handler(event, context):
    resp = sm.start_pipeline_execution(PipelineName='ChurnModelPipeline')
    return {'PipelineExecutionArn': resp['PipelineExecutionArn']}

zip trigger_pipeline.zip trigger_pipeline.py
aws lambda create-function --function-name trigger-churn-pipeline \
  --runtime python3.11 --handler trigger_pipeline.handler \
  --zip-file fileb://trigger_pipeline.zip \
```

```
--role arn:aws:iam::<ACCOUNT_ID>:role/lambda-sagemaker-role --timeout 30
```

Step 3: Create a scheduled EventBridge rule

```
aws events put-rule --name weekly-retrain \
  --schedule-expression 'cron(0 6 ? * MON *)' \
  --description 'Retrain churn model every Monday 06:00 UTC'

aws lambda add-permission --function-name trigger-churn-pipeline \
  --statement-id eventbridge-invoke --action lambda:InvokeFunction \
  --principal events.amazonaws.com \
  --source-arn arn:aws:events:us-east-1:<ACCOUNT_ID>:rule/weekly-retrain

aws events put-targets --rule weekly-retrain \
  --targets 'Id=1,Arn=arn:aws:lambda:us-east-1:<ACCOUNT_ID>:function:trigger-churn-pipeline'
```

Step 4: Test the schedule immediately without waiting a week

Temporarily change the rule to rate(5 minutes), confirm a pipeline execution starts, then switch it back:

```
aws events put-rule --name weekly-retrain --schedule-expression 'rate(5 minutes)'
# wait 5-10 minutes, check:
aws sagemaker list-pipeline-executions --pipeline-name ChurnModelPipeline --max-results 3

# restore the real schedule
aws events put-rule --name weekly-retrain --schedule-expression 'cron(0 6 ? * MON *)'
```

Part B — CloudWatch Dashboards & Alarms

Step 5: Create alarms on endpoint health

```
aws cloudwatch put-metric-alarm \
  --alarm-name churn-endpoint-high-latency \
  --namespace AWS/SageMaker --metric-name ModelLatency \
  --dimensions Name=EndpointName,Value=churn-rt-endpoint \
  Name=VariantName,Value=Primary \
  --statistic Average --period 60 --threshold 500 \
  --comparison-operator GreaterThanThreshold --evaluation-periods 2 \
  --alarm-actions arn:aws:sns:us-east-1:<ACCOUNT_ID>:mlops-alerts

aws cloudwatch put-metric-alarm \
  --alarm-name churn-endpoint-5xx-errors \
  --namespace AWS/SageMaker --metric-name Invocation5XXErrors \
  --dimensions Name=EndpointName,Value=churn-rt-endpoint \
  Name=VariantName,Value=Primary \
  --statistic Sum --period 60 --threshold 5 \
  --comparison-operator GreaterThanThreshold --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:us-east-1:<ACCOUNT_ID>:mlops-alerts
```

Step 6: Build a CloudWatch dashboard

```
aws cloudwatch put-dashboard --dashboard-name ChurnModelOps --dashboard-body '{
```

```

"widgets": [
  {"type": "metric", "x": 0, "y": 0, "width": 12, "height": 6, "properties": {
    "title": "Model Latency",
    "metrics": [{"AWS/SageMaker", "ModelLatency", "EndpointName", "churn-rt-
endpoint", "VariantName", "Primary"}],
    "period": 60, "stat": "Average"}},
  {"type": "metric", "x": 12, "y": 0, "width": 12, "height": 6, "properties": {
    "title": "Invocations & Errors",
    "metrics": [
      ["AWS/SageMaker", "Invocations", "EndpointName", "churn-rt-
endpoint", "VariantName", "Primary"],
      ["AWS/SageMaker", "Invocation5XXErrors", "EndpointName", "churn-rt-
endpoint", "VariantName", "Primary"]
    ],
    "period": 60, "stat": "Sum"}}
]
}'

```

Step 7: Trigger the alarm on purpose

Send a burst of malformed requests (wrong ContentType) to churn-rt-endpoint to generate 5XX errors, then confirm within a few minutes: the CloudWatch alarm transitions to ALARM state, and an email arrives via SNS.

```

import boto3
rt = boto3.client('sagemaker-runtime')
for _ in range(10):
    try:
        rt.invoke_endpoint(EndpointName='churn-rt-endpoint',
        ContentType='application/bad', Body='not-valid-csv')
    except Exception as e:
        print('expected error:', e)

```

Part C — Detecting Data Drift with SageMaker Model Monitor

Step 8: Create a baseline from training data

```

from sagemaker.model_monitor import DefaultModelMonitor
from sagemaker.model_monitor.dataset_format import DatasetFormat

monitor = DefaultModelMonitor(role=role, instance_count=1,
    instance_type='ml.m5.large', volume_size_in_gb=20, max_runtime_in_seconds=1800)

monitor.suggest_baseline(
    baseline_dataset=f's3://{bucket}/data/train/train.csv',
    dataset_format=DatasetFormat.csv(header=True),
    output_s3_uri=f's3://{bucket}/baseline/',
)

```

Step 9: Schedule ongoing monitoring against live traffic

This uses the Data Capture files from the endpoint (enabled in the Deployment & Serving guide, Lab 2) and compares their statistics to the baseline every hour.

```

from sagemaker.model_monitor import CronExpressionGenerator

monitor.create_monitoring_schedule(

```

```

monitor_schedule_name='churn-endpoint-drift-check',
endpoint_input='churn-rt-endpoint',
output_s3_uri=f's3://{bucket}/monitor-reports/',
statistics=monitor.baseline_statistics(),
constraints=monitor.suggested_constraints(),
schedule_cron_expression=CronExpressionGenerator.hourly(),
)

```

Step 10: Review a drift report

After the schedule runs at least once (or trigger it manually via `monitor.run_baseline()` equivalents for testing), inspect results in Studio → Endpoints → churn-rt-endpoint → Monitoring, or list violations directly:

```

executions = monitor.list_executions()
latest = executions[-1]
print(latest.describe()['ProcessingJobStatus'])
violations = latest.constraint_violations()
print(violations.body_dict if violations else 'No violations found')

```

What counts as drift: Model Monitor flags things like a feature's mean shifting outside its baseline range, a categorical feature seeing new unseen values, or missing-value rates increasing — all signs the live population no longer resembles the training population.

Validation Checklist

- EventBridge rule successfully starts a pipeline execution on schedule (verified via list-pipeline-executions)
- CloudWatch dashboard renders both widgets with live data
- Deliberately-triggered errors cause the alarm to fire and an SNS email to arrive
- Model Monitor produces at least one baseline and one monitoring execution with a viewable report

Troubleshooting

Issue	Likely Cause / Fix
EventBridge rule never fires Lambda	Confirm the resource-based Lambda permission (add-permission) was added for <code>events.amazonaws.com</code> — a missing permission fails silently from EventBridge's perspective
No SNS email arrives	Check the subscription status is 'Confirmed', not 'PendingConfirmation', and check spam folder
Monitor schedule shows Failed	Check that Data Capture is actually enabled and producing files — Model Monitor has nothing to compare against without captured request/response data
suggest_baseline fails with ClientError	Verify the training CSV has a header row and the execution role has read access to that S3 path

Challenge Exercise

Chain the two flows together: modify the Lambda from Step 2 so that if a monitoring execution reports violations, it automatically calls `sm.start_pipeline_execution` to kick off retraining immediately, instead of waiting for the weekly schedule — a simple closed-loop retraining trigger.

Final Cleanup — Run This Before Ending the Session

```
aws events remove-targets --rule weekly-retrain --ids 1
aws events delete-rule --name weekly-retrain
aws lambda delete-function --function-name trigger-churn-pipeline
aws cloudwatch delete-alarms --alarm-names churn-endpoint-high-latency churn-
endpoint-5xx-errors
aws cloudwatch delete-dashboards --dashboard-names ChurnModelOps
python -c "from sagemaker.model_monitor import DefaultModelMonitor; import
sagemaker; m=DefaultModelMonitor.attach('churn-endpoint-drift-check',
sagemaker.Session()); m.delete_monitoring_schedule()"
aws sns delete-topic --topic-arn arn:aws:sns:us-east-1:<ACCOUNT_ID>:mlops-alerts
aws sagemaker delete-endpoint --endpoint-name churn-rt-endpoint
```